

FORMATION EMBARQUÉE

Module 09 — Parcours & ressources

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Module 09 — Parcours d'apprentissage et ressources

Un plan réaliste sur 12 mois (à ton rythme : compte les mois en « mois de pratique régulière », ~5-8 h/semaine), les projets jalons, le matériel, et les meilleures ressources gratuites.

1. Le plan sur 12 mois

Phase 1 — Fondations (mois 1-3)

Quoi	Objectif de sortie
Module 00 (fondamentaux)	convertir binaire/hexa de tête, expliquer UART vs I2C vs SPI
Module 01 (C) sur PC	pointeurs, bit à bit, FSM et ring buffer écrits sans aide
Module 03 (Arduino) en parallèle	station météo étapes 1-4 fonctionnelle
Shell + Git (module 06 §4)	chaque projet versionné sur GitHub

Jalon : le thermostat à hystérésis (exercice 3 du module 03) marche, le code est propre, sur GitHub, avec un README.

Phase 2 — Approfondissement (mois 4-6)

Quoi	Objectif de sortie
Module 02 (C++)	station météo refactorée en classes ; RAII compris
Module 03 suite (ESP32)	mesures publiées en MQTT sur le réseau
Module 04 (VHDL) sur simulateur	compteur, FSM, testbenchs GHDL maîtrisés
Module 06 : Make/CMake	un projet compilé sans IDE

Jalon : l'émetteur UART en VHDL (exercice 5 du module 04) simulé et vérifié au chronogramme.

Phase 3 — Industrialisation (mois 7-9)

Quoi	Objectif de sortie
Module 07 (Siemens) avec PLCSIM	station de remplissage + écran WinCC
Module 08 (Schneider) avec Machine Expert Basic	mêmes exercices, plus Modbus
GRAFCET papier	dessiner le GRAFCET AVANT de coder, systématiquement
Module 05 (Java)	superviseur MQTT → SQLite → HTTP du module 05

Jalon : chaîne complète capteur ESP32 → MQTT → Java → base de données, ET un projet automate simulé avec HMI.

Phase 4 — Spécialisation (mois 10-12)

Choisis UNE voie et creuse (c'est ce qui te rend embauchable) :

- **Firmware** : STM32 + HAL + FreeRTOS (module 10 en entier, appuyé sur le module 06 §5), debug SWD, puis refaire un driver en accès registres.
- **FPGA** : carte réelle (Tang Nano/Basys 3), VGA ou filtre audio, timing.
- **Automatisme** : Control Expert + un projet complet avec SFC natif, variateur simulé, supervision — et candidature en alternance/stage.
- **Linux embarqué** : Raspberry Pi sans bureau, GPIO en C via gpiod, service systemd, puis Buildroot.
- **Rust embarqué** : The Rust Book puis refaire la station météo en embassy.

Jalon final : un portfolio GitHub avec 3-4 projets documentés (photos, schémas, README) — ça vaut plus qu'un CV en entretien.

2. Budget matériel récapitulatif

Priorité	Matériel	Prix approx.
1	Arduino Uno/Nano + kit démarrage (breadboard, LED, boutons, capteurs, servo)	25-40 €
2	ESP32 DevKit	8-12 €
3	Modules I2C/SPI : OLED SSD1306, DHT22, HC-SR04, module SD, RTC	15-25 €
4	Multimètre correct	20-30 €
5	STM32 Nucleo-64 (ST-Link intégré)	15-20 €
6	FPGA : Tang Nano 9K (15 €) ou Basys 3 (~150 €, si études)	15-150 €
Bonus	Analyseur logique 8 voies (clone Saleae, +sigrok/PulseView) — magique pour VOIR l'UART/I2C/SPI du module 00	10-15 €

Automates : rien à acheter — simulateurs PLCSIM et Machine Expert Basic. (Un M221 ou S7-1200 d'occasion ~100-200 € seulement si tu veux du réel.)

3. Ressources gratuites recommandées

C / C++

- *Modern C* (Jens Gustedt) — PDF gratuit ; en français : cours C d'OpenClassrooms/Zeste de Savoir.
- learncpp.com — LA référence C++ progressive.
- godbolt.org (Compiler Explorer) — voir l'assembleur généré en direct.
- exercism.org (pistes C et C++) — exercices corrigés.

Arduino / microcontrôleurs

- docs.arduino.cc + le forum français.
- wokwi.com — simulateur en ligne (Uno, ESP32, Pico).
- Chaînes YouTube : GreatScott!, Andreas Spiess (EN) ; U=RI, Tuto Arduino (FR).
- deepbluembedded.com et controllerstech.com — tutoriels STM32.

VHDL / FPGA

- ghdl + GTKWave (open source) ; edaplayground.com (navigateur).
- nandland.com — tutoriels VHDL débutant excellents.
- fpga4student.com, hdlbits (Verilog mais concepts identiques).
- Livre : *Free Range VHDL* (gratuit, PDF).

Java

- dev.java (tutoriels officiels Oracle) ; *Java pour les nuls* si besoin FR.
- baeldung.com — référence pratique.
- JetBrains Academy (parcours gratuit partiel).

Automatisme Siemens / Schneider

- **SCE Siemens** (Siemens Automation Cooperates with Education) : modules de formation TIA Portal gratuits, traduits en français.
- Schneider : Machine Expert Basic gratuit + eLearning Schneider (Université).
- realpars.com (payant mais extraits YouTube gratuits très bons), plcacademy.com.
- FR : chaînes YouTube d'enseignants en BTS CRSA/Électrotechnique ; cours de GRAFCET des académies (PDF librement accessibles).
- Norme à connaître de nom : IEC 61131-3 (langages), IEC 60848 (GRAFCET).

Bas niveau / OS / RTOS

- *The Embedded Rust Book*, *The Rust Book* (gratuits).
- freertos.org — documentation et livre PDF gratuit (*Mastering the FreeRTOS Kernel*).
- bootlin.com — supports de formation Linux embarqué en libre accès (société française).
- *Operating Systems: Three Easy Pieces* — OS expliqués, gratuit.

Pratique et communauté

- github.com — publie tout ; lis le code des bibliothèques que tu utilises.
- electronics.stackexchange.com, forum arduino.cc, r/embedded, r/PLC.
- Concours/défis : Advent of Code (algo en C), hackaday.io (projets).

4. Les erreurs de débutant à éviter

1. **Regarder des tutos sans taper le code.** Ratio sain : 1 h de vidéo → 3 h de pratique.

2. Sauter le C pour aller « direct au concret » Arduino : tu plafonneras vite sans pointeurs ni bits.
 3. Traiter le VHDL comme un langage de programmation (boucles, delays...) : pense circuit, simule tout.
 4. Copier des sketchs entiers sans les relire : recopie à la main, modifie, casse, répare.
 5. Négliger l'électronique de base : la moitié des « bugs logiciels » des débutants sont des masses non reliées, des pull-ups absentes ou des alimentations faiblardes.
 6. Apprendre 6 langages en parallèle : suis les phases, une brique à la fois.
 7. Ne pas versionner : Git dès le premier jour, même pour un blink.
 8. En automatisme : coder avant d'avoir dessiné le GRAFCET et listé les E/S.
-

5. Et pour l'emploi (France) ?

Métiers accessibles avec ce parcours : - **Technicien / ingénieur automaticien** (modules 00, 07, 08 + électrotechnique) — forte demande, partout en France. - **Développeur firmware / systèmes embarqués** (00-03, 06 ; C/C++, STM32, RTOS) — aéronautique, automobile, IoT, médical. - **Ingénieur FPGA** (00, 04 ; VHDL, timing) — défense, spatial, télécoms (souvent bac+5, mais le portfolio parle). - **Développeur IoT full-stack** (03, 05, 06 ; du capteur au cloud). - **Test & validation / banc de test** (C, Python, instruments) — excellente porte d'entrée.

Dans tous les cas : le **portfolio GitHub de projets finis et documentés** est ton meilleur argument, surtout en reconversion.

Bon courage — et souviens-toi : *tout* ce cours se résume à une seule habitude : **construire des petits projets qui marchent, souvent.**